

3

USER-MODE APPLICATIONS



In the previous chapter, we discussed the Windows kernel. But a user doesn't typically interact directly with the kernel. Instead, they interact with user-facing applications, such as word processors and file managers. This chapter will detail how these user-mode applications are created and how they interface with the kernel to provide services to the user.

We'll start by discussing the Win32 application programming interfaces (APIs) designed for user-mode application development and how they relate to the design of the Windows operating system. Then we'll cover the structure of the Windows user interface and how you can inspect it programmatically. Multiple users of a Windows system can all access a user interface at the same time; we'll also look at how console sessions can isolate one user's interface and application resources from those of other users on the same system.

To understand how user-mode applications function, it's important to understand how the provided APIs interface with the underlying kernel system call interface. We'll examine this too, along with the conversion process that filepaths must undergo to become compatible with the kernel. Next, we'll cover how Win32 applications access the registry; then we'll consider how Win32 handles process and thread creation and look at some important system processes.

Win32 and the User-Mode Windows APIs

Most of the code that runs on Windows does not directly interact with system calls. This is an artifact of the *Windows NT* operating system's original design. Microsoft initially developed Windows NT as an updated version of IBM's OS/2 operating system, intending it to have multiple subsystems that implemented different APIs. At various times, it supported POSIX, OS/2, and the Win32 APIs.

Eventually, Microsoft's relationship with IBM went sour, and Microsoft took the API set it had developed for Windows 95, *Win32*, and built a subsystem to implement it. The largely unloved OS/2 subsystem was removed

in Windows 2000, while POSIX survived until Windows 8.1. By Windows 10, Win32 was the only remaining subsystem (though Microsoft subsequently implemented Linux compatibility layers, such as Windows Subsystem for Linux, that don't use the old subsystem extension points).

To allow for these multiple APIs, the Windows kernel implements a generic set of system calls. It's the responsibility of each subsystem's specific libraries and services to convert their APIs to the low-level system call interface. **Figure 3-1** shows an overview of the Win32 subsystem API libraries.

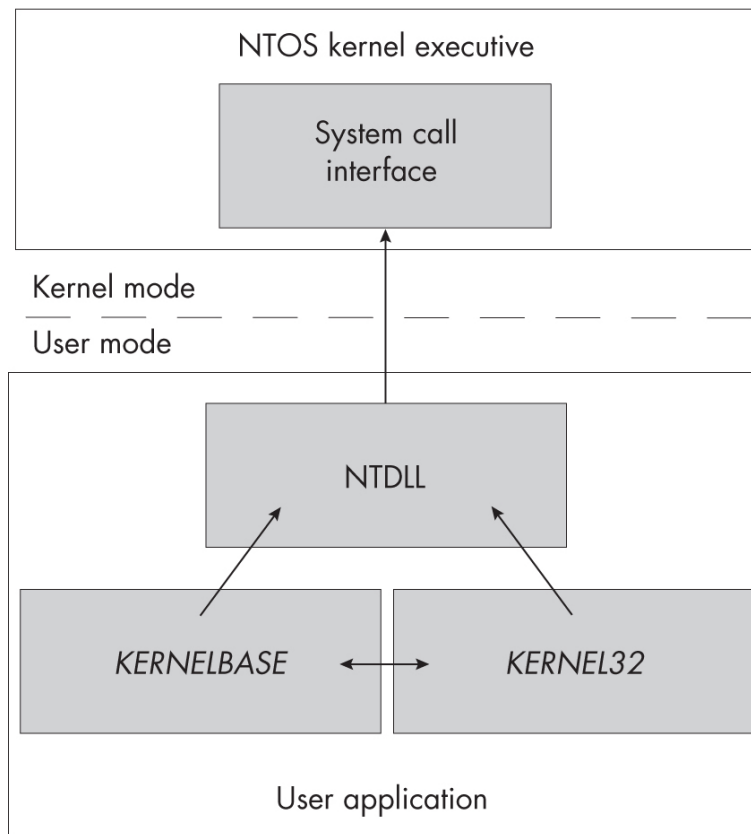


Figure 3-1: The Win32 API modules

As you can see, the core of the Win32 APIs is implemented in the *KERNEL32* and *KERNELBASE* libraries. These libraries call methods in the system-provided *NT Layer dynamic link library (NTDLL)*, which implements system call dispatches as well as runtime library APIs to perform common low-level operations.

Most user-mode applications do not directly contain the implementation of the Windows system APIs. Instead, *NTDLL* includes the DLL loader, which loads new libraries on demand. The loading process is mostly opaque to the developer: when building a program, you link against a set of libraries, and the compiler and toolchain automatically add an import table to your executable file to reflect your dependencies. The DLL loader then inspects the import table, automatically loads any dependent libraries, and resolves the imports. You can also specify exported functions from your application so that other code can rely on your APIs.

Loading a New Library

It's possible to access exported functions manually at runtime without needing an import table entry. You can load a new library using the `LoadLibrary` Win32 API, which is exposed to PowerShell using the `Import-Win32Module` command. To find the memory address of a function exported by a DLL, use the Win32 API `GetProcAddress`, exposed with the PowerShell `Get-Win32ModuleExport` command ([Listing 3-1](#)).

```
❶ PS> $lib = Import-Win32Module -Path "kernel32.dll"
❷ PS> $lib
      Name          ImageBase          EntryPoint
      ----          -
      KERNEL32.DLL  00007FFA088A0000 00007FFA088B7C70

❸ PS> Get-Win32ModuleExport -Module $lib
Ordinal Name                      Address
-----
1       AcquireSRWLockExclusive  NTDLL.RtlAcquireSRWLockExclusive
2       AcquireSRWLockShared    NTDLL.RtlAcquireSRWLockShared
3       ActivateActCtx          0x7FFA088BE640
4       ActivateActCtxWorker    0x7FFA088BA950
--snip--

❹ PS> "{0:X}" -f (Get-Win32ModuleExport -Module $lib
                  -ProcAddress "AllocConsole")
7FFA088C27C0
```

Listing 3-1: Exports for the KERNEL32 library

Here, we use PowerShell to load the `KERNEL32` library and enumerate the exported and imported APIs. First we load it into memory using `Import-Win32Module` ❶. The `KERNEL32` library is always loaded, so this command will just return the existing loaded address; for other libraries, however, the load will cause the DLL to be mapped into memory and initialized.

WARNING

The `Import-Win32Module` command will load a DLL into memory and potentially execute code. In this example, this is acceptable, as `KERNEL32` is one of the trusted system libraries. However, do not use the command on an untrusted DLL, especially if you're analyzing malware, as it might result in malicious code execution. To be safe, always perform malware analysis on a segregated system dedicated to that purpose.

Once it's loaded into memory, we can display some of the library's properties ❷. These include the name of the library, as well as the loaded memory address and the address of the `EntryPoint`. A DLL can optionally define a function, `DllMain`, to run when it's loaded. The `EntryPoint` address is the first instruction in memory to execute when the DLL is loaded.

Next, we dump all exported functions from the DLL ❸. In this case, we see three pieces of information for each: `Ordinal`, `Name`, and `Address`. The `Ordinal` is a small number that uniquely identifies the exported function in the

DLL. It's possible to import an API by its ordinal number, which means there is no need to export a name; you'll see certain names missing from export tables in DLLs whenever Microsoft doesn't want to officially support the function as a public API.

The `Name` is just the name of the exported function. It doesn't need to match what the function was called in the original source code, although typically it does. Finally, `Address` is the address in memory of the function's first instruction. You'll notice that the first two exports have a string instead of an address. This is a case of *export forwarding*; it allows a DLL to export a function by name and has the loader automatically redirect it to another DLL. In this case, `AcquireSRWLockExclusive` is implemented as `RtlAcquireSRWLockExclusive` in *NTDLL*. We can also use `Get-Win32ModuleExport` to look up a single exported function using the `GetProcAddress` API [4](#).

Viewing Imported APIs

In a similar fashion, we can view the APIs that an executable has imported from other DLLs using the `Get-Win32ModuleImport` command, as shown in [Listing 3-2](#).

```
PS> Get-Win32ModuleImport -Path "kernel32.dll"
DllName                               FunctionCount DelayLoaded
-----
api-ms-win-core-rtlsupport-l1-1-0.dll 13           False
ntdll.dll                             378          False
KERNELBASE.dll                        90           False
api-ms-win-core-processthreads-l1-1-0.dll 39          False
--snip--

PS> Get-Win32ModuleImport -Path "kernel32.dll" -DllName "ntdll.dll" |
Where-Object Name -Match "^Nt"
Name                                Address
----
NtEnumerateKey                      7FFA090BC6F0
NtTerminateProcess                  7FFA090BC630
NtMapUserPhysicalPagesScatter       7FFA090BC110
NtMapViewOfSection                  7FFA090BC5B0
--snip--
```

Listing 3-2: Enumerating imports for the *KERNEL32* library

We start by calling `Get-Win32ModuleImport`, specifying the *KERNEL32* DLL as the path. When you specify a path, the command will call `Import-Win32Module` for you and display all the imports, including the name of the DLL to load and the number of functions imported. The final column indicates whether the DLL was marked by the developer as being *delay loaded*. This is a performance optimization; it allows a DLL to be loaded only when one of its exported functions is used. This delay avoids loading all DLLs into memory during initialization, which decreases process startup time and reduces runtime memory usage if the import is never used.

Next, we dump the imported functions for a DLL. As the executable can import code from multiple libraries, we specify the one we want using the `DllName` property. We then filter to all imported functions starting with the `nt` prefix; this allows us to see exactly what system calls *KERNEL32* imports from *NTDLL*.

API SETS

You might notice something odd in the list of imported DLL names in [Listing 3-2](#). If you search your filesystem for the *api-ms-win-core-rtlsupport-l1-1-0.dll* file, you won't find it. This is because the DLL name refers to an API set. *API sets* were introduced in Windows 7 to modularize the system libraries, and they abstract from the name of the set to the DLL that exports the API.

API sets allow an executable to run on multiple different versions of Windows, such as a client, a server, or an embedded version, and change its functionality at runtime based on what libraries are available. When the DLL loader encounters one of these API set names, it consults a table loaded into every process, sourced from the file *apisetschema.dll*, that maps the name to the real DLL. You can query the details for an API set by using the `Get-NtApiSet` command and specifying the name of the API set:

```
PS> Get-NtApiSet api-ms-win-core-rtlsupport-l1-1-0.dll
Name                               HostModule  Flags
----                               -
api-ms-win-core-rtlsupport-l1-1-1  ntdll.dll   Sealed
```

We can see that in this case the API set resolves to the *NTDLL* library. You can also specify the `ResolveApiSet` parameter to the `Get-Win32ModuleImport` command to group the imports based on the real DLLs:

```
PS> Get-Win32ModuleImport -Path "kernel32.dll" -ResolveApiSet
DllName                FunctionCount  DelayLoaded
-----
ntdll.dll               392            False
KERNELBASE.dll          867            False
ext-ms-win-oobe-query-l1-1-0.dll  1              True
RPCRT4.dll              10             True
```

If you compare the output in [Listing 3-2](#) to that of the same command shown here, you'll notice that the resolved imports list is much shorter and that the core libraries have gained additional function imports. Also notice the unresolved API set name, *ext-ms-win-oobe-query-l1-1-0.dll*. Any API set with the prefix `api` should always be present, whereas one with the prefix `ext` might not be. In this case, the API set is not present, and trying to call the imported function will fail. However, because the function is marked as delay loaded, an executable can check whether the API set is available before calling the function by using the `IsApiSetImplemented` Win32 API.

Searching for DLLs

When loading a DLL, the loader creates an image section object from the executable file and maps it into memory. The kernel is responsible for mapping the executable memory; however, user-mode code still needs to parse the import and export tables.

Let's say you pass the string `ABC.DLL` to the `LoadLibrary` API. How does the API know where to find that DLL? If the file hasn't been specified as an absolute path, the API implements a path-searching algorithm. The algorithm, as originally implemented in Windows NT 3.1, searches for files in the following order:

1. The same directory as the current process's executable file
2. The current working directory
3. The Windows *System32* directory
4. The *Windows* directory
5. Each semicolon-separated location in the `PATH` environment variable

The problem with this load order is that it can lead to a privileged process loading a DLL from an insecure location. For example, if a privileged process changed its current working directory using the `SetCurrentDirectory` API to a location a less privileged user could write to, the DLL would be loaded from that location before any DLL from the *System32* directory. This attack is called *DLL hijacking*, and it's a persistent problem on Windows.

Vista changed the default load order to the following, which is safer:

1. The same directory as the current process's executable file
2. The Windows *System32* directory
3. The *Windows* directory
4. The current working directory
5. Each semicolon-separated location in the `PATH` environment variable

Now we no longer load from the current working directory before the *System32* or *Windows* directory. However, if an attacker could write to the executable's directory, a DLL hijack could still take place. Therefore, if an executable is run as a privileged process, only administrators should be able to modify its directory to prevent a DLL hijack from occurring.

THE .DLL FILE EXTENSION

A separate loading quirk involves the handling of file extensions in a DLL's filename. If no extension is specified, the DLL loader will automatically add a *.DLL* extension. If any extension is specified, the filename is treated as is. Finally, if

the extension consists of a single period (for example, *LIB.*), the loader removes the period and tries to load the file without an extension (here, *LIB*).

This file extension behavior can introduce mismatches between the DLL an application is trying to load and the one it actually loads. For example, an application might check that the file *LIB* is valid (that is, correctly cryptographically signed); however, the DLL loader would then load *LIB.DLL*, which was not checked. This can result in security vulnerabilities if you can trick a privileged application into loading the wrong DLL into memory, as the entry point will execute in the privileged context.

While the DLL loader will normally turn to the disk to retrieve a library, some libraries are used so often that it makes sense to pre-initialize them. This improves performance and prevents the DLLs from being hijacked. Two obvious examples are *KERNEL32* and *NTDLL*.

Before any user applications start on Windows, the system configures a *KnownDlls* OMNS directory containing a list of preloaded image sections. A *KnownDlls* Section object's name is just the filename of the library. The DLL loader can check *KnownDlls* first before going to the disk. This improves performance as the loader no longer needs to create a new Section object for the file. It also has a security benefit, ensuring that anything considered to be a known DLL can't be hijacked.

We can list the object directory using the *NtObject* drive, as shown in [Listing 3-3](#).

```
PS> ls NtObject:\KnownDlls
Name                               TypeName
----                               -
kernel32.dll                       Section
kernel.appcore.dll                Section
windows.storage.dll               Section
ucrtbase.dll                      Section
MSCTF.dll                         Section
--snip--
```

Listing 3-3: Listing the contents of the KnownDlls object directory

This section covered the basics of the Win32 subsystem and how it uses libraries to implement the APIs that a user-mode application can use to interface with the operating system. We'll come back to the Win32 APIs later, but first we must discuss the Windows user interface, which is inextricably linked to how the Win32 subsystem functions.

The Win32 GUI

The name "Windows" refers to the structure of the operating system's graphical user interface (GUI). This GUI consists of one or more windows that the user can interact with using controls such as buttons and text input. Since Windows 1.0, the GUI has been the most important feature of

the operating system, so it should come as no surprise that its model is complex. The implementation of the GUI is split between the kernel and user mode, as shown in [Figure 3-2](#).

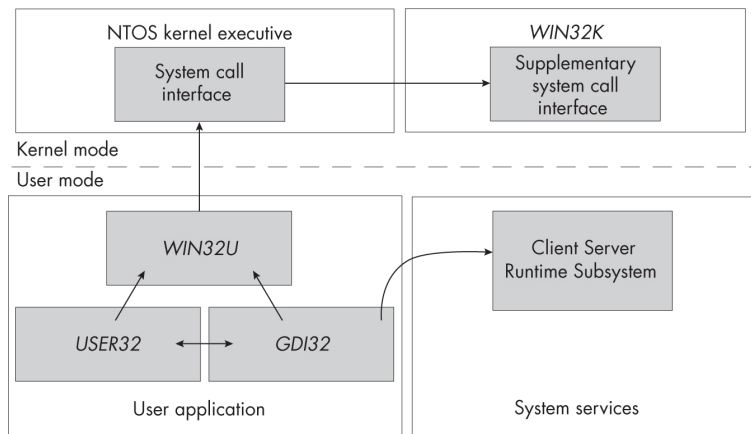


Figure 3-2: The Win32 GUI modules

You might notice that the left-hand side of [Figure 3-2](#) looks a lot like [Figure 3-1](#), which showed the modules for the normal Win32 APIs. In place of *NTDLL*, however, is *WIN32U*, which implements system call stubs for the kernel to call. Two libraries call *WIN32U*: *USER32* and *GDI32*. *USER32* implements the window UI elements and generally manages the GUI, whereas *GDI32* implements drawing primitives, like fonts and shapes.

One big difference between [Figure 3-2](#) and [Figure 3-1](#) is that the GUI is not actually implemented inside the main NTOS kernel executive. Instead, its system calls are implemented in the *WIN32K* driver, which interfaces with the object manager, the kernel, and the display drivers to handle user interactions and display the results. The *WIN32K* driver also implements a system call table that is separate from the kernel's.

NOTE

In versions of Windows prior to 10, the system call dispatch code in WIN32U was embedded directly inside the user-mode DLLs. This made it hard for an application to directly call WIN32K system calls without writing assembly language.

The GUI APIs also interact with a special privileged process: the *Client Server Runtime Subsystem (CSRSS)*. This process is responsible for handling certain privileged operations for lower-privileged clients, such as configuring per-user drive mappings, process management, and error handling.

GUI Kernel Resources

The GUI is made up of four types of kernel resources:

Window stations Objects that represent the connection to the screen and the user interface, such as the keyboard and mouse

Windows GUI elements for interacting with the user, accepting input, and displaying a result

Desktops Objects that represent the visible desktop and act as a host for windows

Drawing resources Bitmaps, fonts, or anything else that needs to be displayed to the user

While the Win32 kernel and user components handle the windows, the window stations and desktops are accessible through the object manager. There are kernel object types for window stations and desktops, as shown in [Listing 3-4](#).

```
PS> Get-NtType WindowStation,Desktop
Name
----
WindowStation
Desktop
```

Listing 3-4: Showing the WindowStation and Desktop type objects

A window station is assigned to a process either at process startup or using the `NtUserSetProcessWindowStation` API. Desktops are assigned on a per-thread basis using `NtUserSetThreadDesktop`. We can query the names of the window stations and desktops with the commands in [Listing 3-5](#).

```
❶ PS> Get-NtWindowStationName
WinSta0
Service-0x0-b17580b$

❷ PS> Get-NtWindowStationName -Current
WinSta0

❸ PS> Get-NtDesktopName
Default
WinLogon

❹ PS> Get-NtDesktopName -Current
Default
```

Listing 3-5: Displaying all the current window stations and desktops

We start by querying the names of all available window stations ❶. In this example, there are two: the default `WinSta0` window station and `Service-0x0-b17580b$`, which another process has created. The ability to create separate window stations allows a process to isolate its GUI interactions from other processes running at the same time. However, `WinSta0` is special, as it is the only object connected to the user's console.

Next, we check what our current window station name is by using the `Current` parameter ❷. We can see we're on `WinSta0`.

We then query for the names of the desktops on our current window station ❷. We see only two desktops: Default and WinLogon. The WinLogon desktop will be visible only if you run the `Get-NtDesktopName` command as an administrator, as it's used solely to display the login screen, which a normal user application shouldn't be able to access. Desktop objects must be opened relative to a window station path; there isn't a specific object directory for desktops. Therefore, the name of the desktop reflects the name of the window station object.

Finally, we check the name of the current thread's desktop ❸. The desktop we're attached to is shown as Default, as that's the only desktop available to normal user applications. We can enumerate the windows created in a desktop using `Get-NtDesktop` and `Get-NtWindow` ([Listing 3-6](#)).

```
PS> $desktop = Get-NtDesktop -Current
PS> Get-NtWindow -Desktop $desktop
Handle ProcessId ThreadId ClassName
-----
66104 11864 12848 GDI+ Hook Window Class
65922 23860 18536 ForegroundStaging
65864 23860 24400 ForegroundStaging
65740 23860 20836 tooltips_class32
--snip--
```

Listing 3-6: Enumerating windows for the current desktop

As you can see, each window has a few properties. First is its *handle*, which is unique to the desktop. This is not the same type of handle we discussed in the preceding chapter for kernel objects; instead, it's a value allocated by the Win32 subsystem.

To function, a window receives *messages* from the system. For example, when you click a mouse button on a window, the system will send a message to notify the window of the click and what mouse button was pressed. The window can then handle the message and change its behavior accordingly. You can also manually send messages to a window using the `SendMessage` and `PostMessage` APIs.

Each message consists of a numeric identifier—such as `0x10`, which represents the message `WM_CLOSE` to close a window—and two additional parameters. The meaning of the two parameters depends on the message. For example, if the message is `WM_CLOSE`, then neither parameter is used; for other messages, they might represent pointers to strings or integer values.

Messages can be sent or posted. The difference between sending and posting a message is that sending waits for the window to handle the message and return a value, while posting just sends the message to the window and returns immediately.

In [Listing 3-6](#), the `ProcessId` and `ThreadId` columns identify the process and thread that created a window using an API such as `CreateWindowEx`. A window has what's called *thread affinity*, which means that only the creating

thread can manipulate the state of the window and handle its messages. However, any thread can send messages to the window. To handle messages, the creating thread must run a *message loop*, which calls the `GetMessage` API to receive the next available message and then dispatches it to the window's message handler callback function using the `DispatchMessage` API. When an application is not running the loop, you might see Windows applications hanging, as without the loop, the GUI cannot be updated.

The final column in [Listing 3-6](#) is the `ClassName`. This is the name of a *window class*, which acts as a template for a new window. When `CreateWindowEx` is called, the `ClassName` is specified and the window is initialized with default values from the template, such as the style of the border or a default size. It's common for an application to register its own classes to handle unique windows. Alternatively, it can use system-defined classes for things like buttons and other common controls.

Window Messages

Let's look at a simple example in [Listing 3-7](#), in which we send a window message to find the caption text for all the windows on the desktop.

```
❶ PS> $ws = Get-NtWindow
❷ PS> $char_count = 2048
   PS> $buf = New-Win32MemoryBuffer -Length ($char_count*2)

❸ PS> foreach($w in $ws) {
    $len = Send-NtWindowMessage -Window $w -Message 0xD -LParam $buf.DangerousGetHandle() -WParam
    $txt = $buf.ReadUnicodeString($len.ToInt32())
    if ($txt.Length -eq 0) {
        continue
    }
    "PID: $($w.ProcessId) - $txt"
}
PID: 10064 - System tray overflow window.
PID: 16168 - HardwareMonitorWindow
PID: 10064 - Battery Meter
--snip--
```

Listing 3-7: Sending the WM_GETTEXT message to all windows on the desktop

First, we enumerate all the windows on the current desktop using the `Get-NtWindow` command ❶. Next, we allocate a memory buffer to store 2,048 characters ❷. Keep in mind that we'll be using this buffer to store 16-bit Unicode characters, so the number of characters must be multiplied by 2 to determine the size in bytes for the buffer.

In a loop ❸, we then send the `WM_GETTEXT` message (which is message number 0xD) to every window to query the window's caption. We need to specify two parameters: `LParam`, which is a pointer to the buffer we allocated, and `WParam`, which is the maximum number of Unicode characters in the buffer. The values passed in these two parameters will be different for different message types. We wait to receive the result of sending the mes-

sage, which indicates the number of characters that were copied into the buffer. We can then read out the caption string and print it to the output, ignoring any windows that have an empty caption.

There is much more to explore in the windowing system, but those details are outside the scope of this book. I recommend Charles Petzold's seminal work on the topic, *Programming Windows*, 5th edition (Microsoft Press, 1998), if you want to know more about the development of Win32 applications. Next, we'll look at how multiple users can use their own user interfaces on the same system through the creation of console sessions.

Console Sessions

The first version of Windows NT allowed multiple users to be authenticated at the same time and each run processes. However, before the introduction of *Remote Desktop Services (RDS)*, it wasn't possible for different interactive desktops to run multiple user accounts concurrently on the same machine. All authenticated users needed to share a single physical console. Windows NT 4 introduced multiple-console support as an optional, server-only feature before it became standard in Windows XP.

RDS is a service on Windows workstations and servers that allows you to remotely connect to the GUI and interact with the system. It's used for remote administration and to provide shared hosting for multiple users on the same network-connected system. In addition, its functionality has been repurposed to support a mechanism that can switch between users on the same system without having to log users out.

To prepare for a new user login to Windows, the session manager service creates a new session on the console. This session is used to organize a user's window station and desktop objects so that they're separate from those belonging to any other user authenticated at the same time. The kernel creates a `Session` object to keep track of resources, and a named reference to the object is stored in the *KernelObjects* OMNS directory. However, the `Session` object is usually only exposed to the user as an integer. There's no randomness to the integer; it's just incremented as each new console session is created.

The session manager starts several processes in this new session before any user logs in. These include a dedicated copy of CSRSS and the Winlogon process, which display the *credentials* user interface and handle the authentication of the new user. We'll dig into the authentication process more in [Chapter 12](#).

The console session that a process belongs to is assigned when the process starts. (Technically, the console session is specified in the access token, but that's a topic for [Chapter 4](#).) We can observe the processes running in each session by running some PowerShell commands, as shown in [Listing 3-8](#).

```
PS> Get-NtProcess -InfoOnly | Group-Object SessionId
Count Name          Group
-----
-----
```

156	0	{, System, Secure System, Registry...}
1	1	{csrss.exe}
1	2	{csrss.exe}
113	3	{csrss.exe, winlogon.exe, fontdrvhost.exe, dwm.exe...}

Listing 3-8: Displaying the processes in each console session using GetNtProcess

Windows has only one physical console, which is connected to the keyboard, mouse, and monitor. However, it's possible to create a new remote desktop over the network by using a client that communicates using the *Remote Desktop Protocol (RDP)*.

It's also possible to switch the user logged on to the physical console; this enables support for the *Fast User Switching* feature in Windows. When the physical console switches to a new user, the previous user is still logged on and running in the background, but you cannot interact with that user's desktop.

Each console session has its own special kernel memory region. Having duplicated resources ensures that the console sessions are separated; this acts as a security boundary. Session number 0 is special, in that it's only for privileged services and system management. It's normally not possible to use a GUI with processes running in this session.

SHATTER ATTACKS

Prior to Windows Vista, both services and the physical console ran in session 0. As any process was able to send window messages to any other process in the same session, this introduced a security weakness called a *shatter attack*. A shatter attack occurs when a normal user can send a window message to a more privileged application in the same session to elevate privileges. For example, the `WM_TIMER` message could accept an arbitrary function pointer that the more privileged application would call when it received the message. A normal user could send this message with a carefully chosen function pointer to enable arbitrary code execution in the context of the privileged application.

Windows Vista mitigated shatter attacks with two related security features that are still present in the latest versions of Windows. The first was *Session 0 Isolation*, which moved the physical console out of session 0 so that a normal user application cannot send messages to services. The second, *User Interface Privilege Isolation (UIPI)*, prevents lower-privileged processes from interacting with windows at higher privileges. Therefore, even if a service creates a window on the user's desktop, the system will reject any messages sent by the user to a privileged service.

Another important feature associated with console sessions is the separation of named objects. In the previous chapter we discussed the *BaseNamedObjects* directory, which is a global location for named objects that provides a means for multiple users to share resources. However, if

multiple users can be logged in to the system at the same time, you could easily get name conflicts. Windows solves this problem by creating a per-console session BNO directory at `|Sessions|<N>|BaseNamedObjects`, where `<N>` is the console session ID. The `|Sessions` directory also contains a directory for the window stations, under `|Sessions|<N>|Windows`, which ensures that window resources, too, are separated. You can list the BNO directory of the current console session with the `NtObjectSession` drive, as shown in [Listing 3-9](#).

```
PS> ls NtObjectSession:\ | Group-Object TypeName
Count Name                               Group
-----
246 Semaphore                            {SM0:10876:304:WilStaging_02_p0h...}
263 Mutant                               {SM0:18960:120:WilError_02,...}
164 Section                             {fd8HWNDDInterface:3092e,...}
159 Event                               {BrushTransitionsCom...}
  4 SymbolicLink                         {AppContainerNamedObjects, Local, Session, Global}
  1 ALPC Port                            {SIPC_{2819B8FF-EB1C-4652-80F0-7AB4EFA88BE4}}
  2 Job                                  {WinlogonAccess, ProcessJobTracker1980}
  1 Directory                             {Restricted}
```

Listing 3-9: The contents of a session's BNO directory

There is no per-console session BNO for session 0; it uses the global BNO directory.

THE ORIGINS OF REMOTE DESKTOP SERVICES

The RDS feature didn't originate at Microsoft. Rather, a company called Citrix developed the technology for Windows and licensed it to Microsoft for use in NT 4. The technology was originally called Terminal Services, so it's common to sometimes see it referred to using that name. To this day, it's possible to buy a Citrix version of RDS that uses a different network protocol, Independent Computing Architecture (ICA), instead of Microsoft's RDP.

Comparing Win32 APIs and System Calls

Not all system calls are directly exposed through Win32, and in some cases, the Win32 API reduces the functionality of exposed system calls. In this section, we'll look at some common differences between system calls and their Win32 API equivalents.

As a case study, we'll consider the `CreateMutexEx` API, the Win32 version of the `NtCreateMutant` system call we looked at in the preceding chapter. The API has the C prototype shown in [Listing 3-10](#).

```
HANDLE CreateMutexEx(
    SECURITY_ATTRIBUTES* lpMutexAttributes,
    const WCHAR*         lpName,
    DWORD                dwFlags,
    DWORD                dwDesiredAccess
);
```

Listing 3-10: The prototype for the `CreateMutexEx` Win32 API

Compare it to the `NtCreateMutant` prototype, shown in [Listing 3-11](#).

```
NTSTATUS NtCreateMutant(
    HANDLE*      MutantHandle,
    ACCESS_MASK  DesiredAccess,
    OBJECT_ATTRIBUTES* ObjectAttributes,
    BOOLEAN      InitialOwner
);
```

Listing 3-11: The prototype for the `NtCreateMutant` system call

The first difference between the prototypes is that the Win32 API returns a handle to the kernel object, while the system call returns an `NTSTATUS` code (and receives the handle via a pointer as the first parameter instead).

You might wonder: How do errors get propagated back to an API's caller, if not via an `NTSTATUS` code? In this respect, the Win32 APIs are not always consistent. If the API returns a handle, then it's common to return a value of `NULL`. However, some APIs, such as the file APIs, return the value `-1` instead. If a handle is not returned, it's common to return a Boolean value, with `TRUE` indicating success and `FALSE` indicating an error.

But what if we want to know *why* the API failed? For this purpose, the APIs define a set of error codes. Unlike the `NTSTATUS` codes, these error codes don't have any structure; they're just numbers. When a Windows API fails, you can query for this error code by calling the `GetLastError` API.

`NTDLL` provides an `RtlNtStatusToDosError` API to convert an `NTSTATUS` code to a predefined Win32 error code. The `CreateMutexEx` API can convert the `NTSTATUS` code to a Win32 error code on failure, then write it to the last error location for the current thread using the `SetLastError` API.

We can look up error codes in PowerShell using `Get-Win32Error`, as shown in [Listing 3-12](#).

```
PS> Get-Win32Error 5
ErrorCode Name          Message
-----
5 ERROR_ACCESS_DENIED Access is denied.
```

Listing 3-12: Looking up Win32 error code 5

The second big difference between the system call and the Win32 API is that the API does not take the `OBJECT_ATTRIBUTES` structure. Instead, it splits the attributes between two parameters: `lpName`, used to specify the object's name, and `lpMutexAttributes`, which is a pointer to a `SECURITY_ATTRIBUTES` structure.

The `lpName` parameter is a NUL-terminated string composed of 16-bit Unicode characters. Even though the object manager uses the counted

UNICODE_STRING, the Win32 API uses a C-style terminated string. This means that while the NUL character is a valid character for an object name, it's impossible to specify using the Win32 API.

Another difference is that the name is not a full path to the OMNS location for the object; instead, it's relative to the current session's BNO directory. This means that if the name is *ABC*, then the final path used is `|Sessions|<N>|BaseNamedObjects|ABC`, where *<N>* is the console session ID. If you want to create an object in the global BNO directory, you can prefix the name with *Global* (for example, *Global|ABC*). This works because *Global* is a symbolic link to *|BaseNamedObjects|*, which is automatically created along with the per-session BNO directory. If you want to simulate this behavior using the `Get` and `New` PowerShell commands, pass them the `-Win32Path` option, as shown in [Listing 3-13](#).

```
PS> $m = New-NtMutant ABC -Win32Path
PS> $m.FullPath
\Sessions\2\BaseNamedObjects\ABC
```

Listing 3-13: Creating a new Mutant with -Win32Path

[Listing 3-14](#) shows the `SECURITY_ATTRIBUTES` structure.

```
struct SECURITY_ATTRIBUTES {
    DWORD    nLength;
    VOID*    lpSecurityDescriptor;
    BOOL     bInheritHandle;
};
```

Listing 3-14: The SECURITY_ATTRIBUTES structure

This allows you to specify the security descriptor of the new object, as well as whether the handle should be inheritable. The `CreateMutexEx Win32` API exposes no other options from `OBJECT_ATTRIBUTES`.

This brings us to the final two parameters in [Listing 3-10](#): `dwDesiredAccess` directly maps to `DesiredAccess`, and the native `InitialOwner` parameter is specified through `dwFlags` with the `CREATE_MUTEX_INITIAL_OWNER` flag.

One surprise you might encounter may occur if you try to look up the address of the `CreateMutexEx` API in the export table of the *KERNEL32* DLL ([Listing 3-15](#)).

```
PS> Get-Win32ModuleExport "kernel32.dll" -ProcAddress CreateMutexEx
Exception calling "GetProcAddress" with "2" argument(s):
"(0x8007007F) - The specified procedure could not be found."
```

Listing 3-15: Getting CreateMutexEx from KERNEL32

Instead of receiving the address, we get an exception. Did we pick the wrong library? Let's try to find the API by dumping all exports and filtering them by name, as shown in [Listing 3-16](#).


```
PS> Get-Win32ModuleExport "kernel32.dll" | Where-Object Name -Match CreateMutexEx
```

Ordinal	Name	Address
217	CreateMutexExA	0x7FFA088C1EB0
218	CreateMutexExW	0x7FFA088C1EC0

Listing 3-16: Finding the CreateMutexEx API by listing all exports

As you can see, the `CreateMutexEx` API is there not once, but twice. Each function has a suffix, either `A` or `W`. This is because Windows 95 (where most of the APIs were initially created) didn't natively support Unicode strings, so the APIs used single-character strings in the current text encoding. With the introduction of Windows NT, the kernel became 100 percent Unicode, but it provided two APIs for a single function to enable older Windows 95 applications.

APIs with an `A` suffix accept single-character strings, or *ANSI strings*. These APIs convert their strings into Unicode strings to pass to the kernel, and they convert them back again if a string needs to be returned. Applications built for Windows NT, on the other hand, can use the APIs with the `W` suffix, for *wide string*; these don't need to do any string conversions. Which API you get when you build a native application depends on your build configuration and is a topic for a completely different book.

Win32 Registry Paths

In [Chapter 2](#), you learned the basics of how to access the registry with native system calls using paths in the OMNS. The Win32 APIs used to access the registry, such as `RegCreateKeyEx`, do not expose these OMNS paths. Instead, you access registry keys relative to predefined root keys. You'll be familiar with these keys if you've ever used the Windows `regedit` application, shown in [Figure 3-3](#).

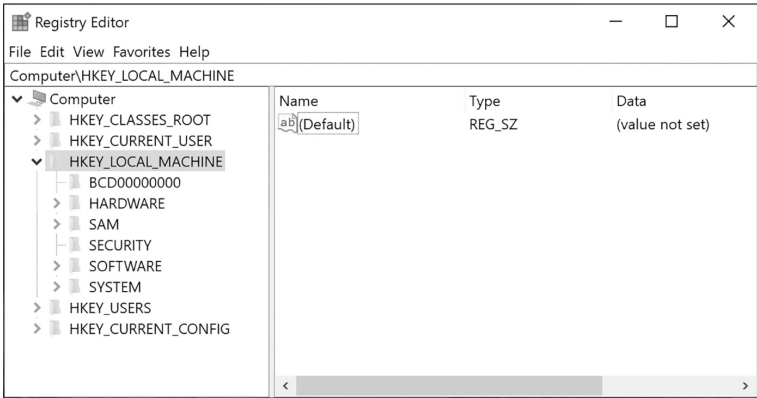


Figure 3-3: The main view of the regedit utility

The handle values displayed in [Figure 3-3](#) are listed in [Table 3-1](#) along with their corresponding OMNS paths.

Table 3-1: Predefined Registry Handles and Their Native Equivalents

Predefined handle name	OMNS path
HKEY_LOCAL_MACHINE	<code>\REGISTRY\MACHINE</code>
HKEY_USERS	<code>\REGISTRY\USER</code>
HKEY_CURRENT_CONFIG	<code>\REGISTRY\MACHINE\SYSTEM\CurrentControlSet\Hardware Profiles\Current</code>
HKEY_CURRENT_USER	<code>\REGISTRY\USER\<SDDL SID></code>
HKEY_CLASSES_ROOT	Merged view of <code>\REGISTRY\MACHINE\SOFTWARE\Classes</code> and <code>\REGISTRY\USER\<SDDL SID>\Classes</code>

The first three predefined handles, `HKEY_LOCAL_MACHINE`, `HKEY_USERS`, and `HKEY_CURRENT_CONFIG`, are not particularly special; they directly map to a single OMNS registry key path. The next handle, `HKEY_CURRENT_USER`, is more interesting; it maps to a hive loaded for the currently authenticated user. The name of the hive's key is the SDDL string of the user's SID.

The final key, `HKEY_CLASSES_ROOT`, which stores information such as file extension mappings, is a merged view of a user's classes hive and the machine's hive. The user's hive takes precedence over the machine's, allowing the user to change their file extensions without needing an administrator.

Opening Keys

When using the `Get-NtKey` and `New-NtKey` commands, we can specify a Win32 path by using the `Win32Path` parameter ([Listing 3-17](#)).

```
PS> Use-NtObject($key = Get-NtKey \REGISTRY\MACHINE\SOFTWARE) {
    $key.Win32Path
}
HKEY_LOCAL_MACHINE\SOFTWARE

PS> Use-NtObject($key = Get-NtKey -Win32Path "HKCU\SOFTWARE") {
    $key.FullPath
}
\REGISTRY\USER\S-1-5-21-818064985-378290696-2985406761-1002\SOFTWARE
```

Listing 3-17: Interacting with the registry using Win32 paths

We start by opening a key object using the `Get-NtKey` command. We use the OMNS path to open the key, then convert the path to its Win32 version using the `Win32Path` property. In this case, we see that `\REGISTRY\MACHINE\SOFTWARE` is mapped to `HKEY_LOCAL_MACHINE\SOFTWARE`.

We then do the reverse and open a key using a Win32 name by specifying the `Win32Path` parameter and printing its native OMNS path. Here, we use the current user's hive. Notice we're using a shortened form of the predefined key name: `HKCU`, instead of `HKEY_CURRENT_USER`. All the other predefined keys have similar shortened forms; for example, `HKLM` refers to `HKEY_LOCAL_MACHINE`.

In the output, you can see the SDDL SID string, which represents the current user. As this example demonstrates, using the Win32 path to access the current user's hive is much simpler than looking up the current user's SID and opening it with the OMNS path.

Listing the Registry's Contents

In the previous chapter, you saw how to list the registry's contents using the `NtObject` or `NtKey` drive provider path. For the Win32 registry, you have a few additional options. To simplify accessing the current user's hive, you can use `NtKeyUser`. For example, you can list the current user's software key with the following:

```
PS> ls NtKeyUser:\SOFTWARE
```

PowerShell also comes with built-in drives, `HKLM` and `HKCU`, for the local machine and current user's hives, respectively. For example, the equivalent to the previous command is the following:

```
PS> ls HKCU:\SOFTWARE
```

Why would you use one of these drive providers over another? Well, the PowerShell module's drive providers have the advantage of allowing you to view the entire registry. They also use the native APIs, which use counted strings and support the use of NUL characters in the names of the registry keys and values. In contrast, the Win32 APIs use NUL-terminated C-style strings, which cannot handle embedded NUL characters. Therefore, if a NUL is embedded into a name, it's impossible for the built-in provider to access that key or value. [Listing 3-18](#) demonstrates this.

```
❶ PS> $key = New-NtKey -Win32Path "HKCU\ABC`0XYZ"
❷ PS> Get-Item "NtKeyUser:\ABC`0XYZ"
Name      TypeName
----      -
ABC XYZ   Key

❸ PS> Get-Item "HKCU:\ABC`0XYZ"
Get-Item : Cannot find path 'HKCU:\ABC XYZ' because it does not exist.

PS> Remove-NtKey $key
PS> $key.Close()
```

Listing 3-18: Adding and accessing a registry key with a NUL character

We start by creating a new key with a NUL character in the name, indicated by the `\0` escape ❶. If you access this path via the *NtKeyUser* drive, you can successfully retrieve the key ❷. However, if you try this with the built-in drive provider, it doesn't work; it can't find the registry key ❸.

This behavior of the Win32 APIs can lead to security issues. For example, it's possible for malicious code to hide registry keys and values from any software that uses the Win32 APIs by embedding NUL characters in the name. This can prevent the malicious code from being detected. We'll see how to uncover the use of this hiding technique in "Finding Hidden Registry Keys or Values" on page 94.

It's also possible to get a mismatch if some software uses the native system calls and other software uses the Win32 APIs. For example, if some code checks the *ABC\0XYZ* path to ensure it has been correctly set up, then hands this to another application, which uses the path with the Win32 APIs, the new application will instead access the unrelated *ABC* key, which hasn't been checked. This could lead to information disclosure issues if the contents of *ABC* were returned to the caller.

The built-in registry provider does have an advantage too: it can be used without the installation of an external module. It also allows you to create new keys and add values, which the module's provider does not allow you to do.

DOS Device Paths

Another big difference between the Win32 APIs and the native system calls is how they handle filepaths. In the previous chapter, we saw that we can access a mounted filesystem using a *Device\<VolumeName>* path. However, we can't specify this native path using the Win32 APIs. Instead, we use well-known paths, such as *C:\Windows*, that have drive letters. Because the drive letter paths are a vestige of MS-DOS, we call them *DOS device paths*.

Of course, the Win32 API needs to pass the system call a native path for the system call to work correctly. The *NTDLL* API *RtlDosPathNameToNtPathName* handles this conversion process. This API takes a DOS device path and returns the fully converted native path. The simplest conversion occurs when the caller has supplied a full drive path: for example, *C:\Windows*. In these cases, the conversion process merely prefixes the path with the predefined path component *\\?* to get the result *\\?\\C:\Windows*.

The *\\?* path, also called the *DOS device map prefix*, indicates that the object manager should use a two-step lookup process to find the drive letter. The object manager will first check a per-user DOS device map directory, in the path *Sessions\0\DosDevices\<AUTHID>*. Because the object manager checks a per-user location first, each user can create their own drive mappings. The *<AUTHID>* component is related to the authentication session of the caller's *Token*; I'll describe this in [Chapter 4](#), but for now, it's enough to know that its value is unique for each user. Note that the use of *0* for the console session ID is not a typo: all DOS device mappings are

placed in a single location, regardless of which console session the user is logged in to.

If the drive letter is not found in the per-user location, the object manager will check a global directory, *GLOBAL??*. If it's not found there, then the file lookup fails. The drive letter is an object manager symbolic link that points to the mounted volume device. We can see this in action by using the `Get-NtSymbolicLink` command to open the drive letters and display their properties ([Listing 3-19](#)).

```
PS> Use-NtObject($cdrive = Get-NtSymbolicLink "\\??\\C:") {
    $cdrive | Select-Object FullPath, Target
}
FullPath      Target
-----
❶ \\GLOBAL??\\C: \\Device\\HarddiskVolume3

❷ PS> Add-DosDevice Z: C:\\Windows
PS> Use-NtObject($zdrive = Get-NtSymbolicLink "\\??\\Z:") {
    $zdrive | Select-Object FullPath, Target
}
FullPath                                     Target
-----
❸ \\Sessions\\0\\DosDevices\\00000000-011b224b\\Z: \\??\\C:\\windows

❹ PS> Remove-DosDevice Z:
```

Listing 3-19: Displaying the symbolic links for the C: and Z: drives

First, we open the C: drive symbolic link and display its `FullPath` and `Target` properties. The full path is in the *GLOBAL??* directory, and the target is the volume path ❶. We then create a new Z: drive using the `Add-DosDevice` command, pointing the drive to the *Windows* directory ❷. Note that the Z: drive is accessible in any user application, not just in PowerShell. Displaying the Z: drive's properties reveals that it's in the per-user DOS device map and that the target is the native path to the *Windows* directory ❸. This shows that the target of a drive letter doesn't have to point directly to a volume, as long as it gets there eventually (in this case, after following the C: drive symbolic link). Finally, for completeness, we remove the Z: drive with `Remove-DosDevice` ❹.

Path Types

[Table 3-2](#) shows several different path types that the Win32 APIs support, along with example native paths after conversion.

Table 3-2: Win32 Path Types

DOS path	Native path	Description
<i>some\path</i>	<i>\\??\\C:\\ABC\\some\path</i>	Relative path to current directory

DOS path	Native path	Description
<code>C:\some\path</code>	<code>\\?\\C:\\some\path</code>	Absolute path
<code>C:\some\path</code>	<code>\\?\\C:\\ABC\\some\path</code>	Drive relative path
<code>\\some\path</code>	<code>\\?\\C:\\some\path</code>	Rooted to current drive
<code>\\.\C:\\some\..\path</code>	<code>\\?\\C:\\path</code>	Device path, canonicalized
<code>\\?\\C:\\some\..\path</code>	<code>\\?\\C:\\some\..\path</code>	Device path, non-canonicalized
<code>\\?\\C:\\some\path</code>	<code>\\?\\C:\\some\path</code>	Device path, non-canonicalized
<code>\\server\share\path</code>	<code>\\?</code> <code>\\UNC\\server\share\path</code>	UNC path to share on server

Due to the way DOS paths are specified, multiple DOS paths might represent the same native path. To ensure the final native path is correct, the DOS path must go through a *canonicalization* process to convert these different representations into the same canonical form.

One simple operation undertaken in canonicalization is the handling of path separators. For native paths, there is only one path separator, the backslash (\) character. If you use a forward slash (/), the object manager will treat it as just another filename character. However, DOS paths support both forward slashes and backslashes as path separators. The canonicalization process takes care of this by ensuring all forward slashes are converted to backslashes. Therefore, `C:\\Windows` and `C:/Windows` are equivalent.

Another canonicalization operation is the resolving of parent directory references. When writing a DOS path, you might specify a filename with one dot (.) or two dots (..), each of which has a special meaning. A single dot refers to the current directory, and the canonicalization process will remove it from the path. A double dot refers to the parent, so the parent directory will be removed. Therefore, the path `C:\\ABC\\XYZ` will get converted to `C:\\ABC\\XYZ`, and `C:\\ABC\\..\\XYZ` will get converted to `C:\\XYZ`. As with the forward slash, the native APIs do not know about these special filenames and will assume that they're the names of the file to look up.

NOTE

Most other operating systems, such as Linux, handle this canonicalization process in the kernel. However, due to the subsystem model, Windows must do the path canonicalization in user mode, inside the subsystem-specific library. This is to support any differences in behavior in OS/2 and POSIX environments.

If the DOS path is prefixed with `||?` or `|??`, then the path is not canonicalized and is instead used verbatim, including any parent directory references or forward slashes. In some cases, the `|??` prefix can confuse the Win32 APIs with a current drive-rooted path, resulting in the opening of a path such as `|??C:|??Path`. It's unclear why Microsoft added this DOS path type, considering its potential for confusion.

You can manually convert a Win32 path to a native path using the `Get-NtFilePath` command. You can also check the path type using the `Get-NtFilePathType` command. [Listing 3-20](#) shows some examples of using these commands.

```
PS> Set-Location $env:SystemRoot
PS C:\Windows> Get-NtFilePathType "."
Relative

PS C:\Windows> Get-NtFilePath "."
\??\C:\Windows

PS C:\Windows> Get-NtFilePath "..\"
\??\C:\

PS C:\Windows> Get-NtFilePathType "C:ABC"
DriveRelative

PS C:\Windows> Get-NtFilePath "C:ABC"
\??\C:\Windows\ABC

PS C:\Windows> Get-NtFilePathType "\\?\C:\abc\..\xyz"
LocalDevice

PS C:\Windows> Get-NtFilePath "\\?\C:\abc\..\xyz"
\??\C:\abc\..\xyz
```

Listing 3-20: Examples of Win32 filepath conversion

When you're using the `Get-NtFile` or `New-NtFile` command, you can use the `Win32Path` property to treat the path as a Win32 path and automatically convert it.

Maximum Path Lengths

The maximum filename length supported by Windows is limited by the maximum number of characters that can be stored in a `UNICODE_STRING` structure (32,767). However, Win32 APIs have a stricter requirement. By default, as shown in [Listing 3-21](#), any attempt to pass a path longer than the value of `MAX_PATH`, defined as 260 characters, will fail. This behavior is implemented inside the `NTDLL` API `RtlDosPathNameToNtPathName` when converting the path from Win32 to native format.

```
PS> $path = "C:\$( 'A'*256 )"
PS> $path.Length
259

PS> Get-NtFilePath -Path $path
```

```

\??\C:\AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

PS> $path += "A"
PS> $path.Length
260

PS> Get-NtFilePath -Path $path
Get-NtFilePath : "(0xC0000106) - A specified name string is too long..."

PS> $path = "\\?\\" + $path
PS> $path.Length
264

PS> Get-NtFilePath -Path $path
\??\C:\AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

```

Listing 3-21: Testing the Win32 MAX_PATH path limit

We call the `RtlDosPathNameToNtPathName` API via the `Get-NtFilePath` command. The first path we create is 259 characters long, which we can successfully convert to a native path. We then add one more character to the path, making the path 260 characters long; this attempt fails with the error `STATUS_NAME_TOO_LONG`. If `MAX_PATH` is 260, you may be wondering: Shouldn't a 260-character-long path succeed? Unfortunately, no. The APIs include the NUL-terminating character as part of the path's length, so the maximum path length is really only 259 characters.

Listing 3-21 also shows a way of bypassing this limitation. If we add the device prefix `\\?` to the path, the conversion succeeds even though the length of the path is now 264 characters. This is because the prefix is replaced with the DOS device prefix `\\?\\`, and the remaining path is left verbatim. While this technique works, note that it also disables useful features, such as path canonicalization. As another workaround, in current versions of Windows there is a way of opting into long filenames, as shown in **Listing 3-22**.

```

PS> $path = "HKLM\SYSTEM\CurrentControlSet\Control\FileSystem"
PS> Get-NtKeyValue -Win32Path $path -Name "LongPathsEnabled"
Name           Type DataObject
----
LongPathsEnabled Dword 1

PS> (Get-Process -Id $pid).Path | Get-Win32ModuleManifest |
Select-Object LongPathAware
LongPathAware
-----
    True

② PS> $path = "C:\$( 'A'*300 )"
PS> $path.Length
303

PS> Get-NtFilePath -Path $path
\??\C:\AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

```

Listing 3-22: Checking and testing long, path-aware applications

The first thing we do here is verify that the `LongPathsEnabled` registry value is set to 1. The value must be set to 1 before the process starts, as it will be read only once during process initialization. However, just enabling the long path feature isn't sufficient: the process's executable file must opt in by specifying a manifest property. We can query this property by using the `Get-ExecutableManifest` command and selecting `LongPathAware`.

Fortunately, PowerShell has this manifest option enabled ❶. We can now convert much larger paths successfully, as shown with a 303-character path ❷.

Are long paths a security issue? It's common for security issues to be introduced in places where there is an interface boundary. In this case, the fact that a filesystem can support exceptionally long paths could lead to the incorrect assumption that a filepath can never be longer than 260 characters. A possible issue might occur when an application queries the full path to a file and then copies that path into a memory buffer with a fixed size of 260 characters. If the length of the filepath is not first checked, this operation could result in the corruption of memory after the buffer, which might allow an attacker to gain control of the application's execution.

Process Creation

Processes are the main way to execute user-mode components and isolate them for security purposes, so it's important that we explore how to create them in detail. In the previous chapter, I mentioned that you can create a process using the `NtCreateUserProcess` system call. However, most processes won't be created directly using this system call; rather, they'll be created with the Win32 `CreateProcess` API, which acts as a wrapper.

The system call isn't often used directly, because most processes need to interact with other user-mode components, especially CSRSS, to interact with the user's desktop. The `CreateProcess` API will register the new process created by the system call with the appropriate services necessary for correct initialization. We won't discuss process and thread creation in detail in this book, but in this section I'll give a quick overview.

Command Line Parsing

The simplest way to create a new process is to specify a command line string representing the executable to run. The `CreateProcess` API will then parse the command line to find the executable file to pass to the kernel.

To test this command line parsing, let's create a new process using the `New-Win32Process` PowerShell command, which executes `CreateProcess` under the hood. We could use a built-in command such as `Start-Process` to do this, but `New-Win32Process` is useful because it exposes the full set of the `CreateProcess` API's functionality. We can start a process using the following command:

```
PS> $proc = New-Win32Process -CommandLine "notepad test.txt"
```

We provide a command line containing the name of the executable to run, Notepad, and the name of a file to open, *test.txt*. This string doesn't necessarily need to provide a full path to the executable; the `New-Win32Process` command will parse the command line to try to distinguish the name of the initial executable image file from the file to open. That's not as simple a process as it sounds.

The first thing `New-Win32Process` will do is parse the command line using an algorithm that splits on whitespace, unless that whitespace is enclosed in double quotes. In this case, it will parse the command line into two strings, `notepad` and `test.txt`. The command then takes the first string and tries to find a matching process. However, there's a slight complication: there is no *notepad* executable file, only *notepad.exe*. Though it's not required, Windows executables commonly have a *.exe* extension, so the search algorithm will automatically append this extension if one doesn't already exist.

The command will then search the following locations for the executable, much like the DLL path searching we discussed in "Searching for DLLs" on page 68. Note that the executable search path is the same as the unsafe DLL search path:

1. The same directory as the current process's executable file
2. The current working directory
3. The Windows *System32* directory
4. The *Windows* directory
5. Each semicolon-separated location in the `PATH` environment variable

If `New-Win32Process` can't find *notepad.exe*, it will next try to find the file *notepad test.txt*, in case that's what we meant. As the filename has an extension already, it won't replace it with *.exe*. If `New-Win32Process` can't find the file, it returns an error. Note that if we passed `notepad` surrounded by double quotes, as in `"notepad" test.txt`, then `New-Win32Process` would search for *notepad.exe* only and never fall back to trying all combinations of the name with the whitespace.

This command line parsing behavior has two security implications. First, if the process is being created by a more privileged process and a less privileged user can write a file to a location earlier in the path search list, then the process could be hijacked.

The second security implication is that the path-searching algorithm changes if the first value contains a path separator. In this case, instead of using the path-searching rules, `New-Win32Process` splits the path by whitespace and then tries each component as if it were a path, searching for the name either with the *.exe* extension or without it.

Let's look at an example. If we specify a command line of `C:\Program Files\abc.exe`, then the following paths will be searched for the executable

file:

- `C:\Program`
- `C:\Program.exe`
- `C:\Program Files\abc.exe`
- `C:\Program Files\abc.exe.exe`

If the user could write the file `C:\Program` or `C:\Program.exe`, then they could hijack execution. Fortunately, on a default installation of Windows, a normal user can't write files to the root of the system drive; however, configuration changes sometimes allow this. Also, the executable path might be on a different drive that does allow writing to the root.

To avoid both security implications, the caller can specify the executable's full pathname by setting the `ApplicationName` property when calling `New-
Win32Process`:

```
PS> $proc = New-Win32Process -CommandLine "notepad test.txt"  
-ApplicationName "C:\windows\notepad.exe"
```

If we specify the path this way, the command will pass it verbatim to the new process.

Shell APIs

If you double-click a non-executable file type, such as a text document, in Explorer, it will helpfully start an editor for you. However, if you try to run a document with `New-Win32Process`, you'll get the error shown here:

```
PS> New-Win32Process -CommandLine "document.txt"  
Exception calling "CreateProcess": "%1 is not a valid Win32 application"
```

This error indicates that the text file is not a valid Win32 application.

The reason Explorer can start the editor is that it doesn't use the underlying `CreateProcess` API directly; instead, it uses a shell API. The main shell API used to start the editor for a file is `ShellExecuteEx`, implemented in the `SHELL32` library. This API and its simpler sibling, `ShellExecute`, are much too complex to cover in detail here. Instead, I'll give just a brief overview of the latter.

For our purposes, we need to specify three parameters to `ShellExecute`:

- The path to the file to execute
- The verb to use on the file
- Any additional arguments

The first thing `ShellExecute` does is look up the handler for the extension of the file to execute. For example, if the file is `test.txt`, then it needs to look up the handler for the `.txt` extension. The handlers are registered in the registry under the `HKEY_CLASSES_ROOT` key, which, as we saw earlier in the

chapter, is a merged view of parts of the machine software and the user’s registry hive. In [Listing 3-23](#), we query the handler.

```
PS> $base_key = "NtKey:\MACHINE\SOFTWARE\Classes"
❶ PS> Get-Item "$base_key\.txt" | Select-Object -ExpandProperty Values
Name      Type      DataObject
----      -
Content Type String text/plain
PerceivedType String text
                ❷ String txtfile

❸ PS> Get-ChildItem "$base_key\txtfile\Shell" | Format-Table
Name      TypeName
----      -
open      Key
print     Key
printto   Key

❹ PS> Get-Item "$base_key\txtfile\Shell\open\Command" |
Select-Object -ExpandProperty Values | Format-Table
Name Type      DataObject
---- -
❺ ExpandString %SystemRoot%\system32\NOTEPAD.EXE %1
```

Listing 3-23: Querying the shell handler for .txt files

We start by querying the machine class’s key for the .txt extension ❶. Although we could have checked for a user-specific key, checking the machine class’s key ensures that we inspect the system default. The .txt registry key doesn’t directly contain the handler. Instead, the default value, represented by an empty name, refers to another key: in this case, the txtfile ❷. We then list the subkeys of txtfile and find three keys: open, print, and printto ❸. We can pass these verbs by name to ShellExecute.

Each of these verb keys can have a subkey, called Command, that contains a command line to execute ❹. We can see that the default for a .txt file is to open Notepad ❺; the %1 is replaced with the path to the file being executed. (The command could also contain %, which includes any additional arguments passed to ShellExecute.) The CreateProcess API can now start the executable and handle the file.

There are many different standard verbs you can pass to ShellExecute. [Table 3-3](#) shows a list of common ones you’ll encounter.

Table 3-3: Common Shell Verbs

Verb	Description
open	Open the file; this is typically the default.
edit	Edit the file.
print	Print the file.

Verb	Description
printto	Print to a specified printer.
explore	Explore a directory; this is used to open a directory in an Explorer window.
runas	Open the file as an administrator; typically, defined for executables only.
runasuser	Open the file as another user; typically, defined for executables only.

You might find it odd that there is both an `open` and an `edit` verb. If you opened a `.txt` file, for example, the file would open in Notepad, and you'd be able to edit it. But the distinction is useful for files such as batch files, where the `open` verb would execute the file and `edit` would open it in a text editor.

To use `ShellExecute` from PowerShell, you can run the `Start-Process` command. By default, `ShellExecute` will use the `open` verb, but you can specify your own verb using the `Verb` parameter. In the following code, we print a `.txt` file as an administrator using the `print` verb:

```
PS> Start-Process "test.txt" -Verb "print"
```

Verb configurations can also improve security. For example, PowerShell scripts with a `.ps1` extension have the `open` verb registered. However, clicking a script will open the script file in Notepad rather than executing the script. Therefore, if you double-click the script file in Explorer, it won't execute. Instead, you must right-click the file and explicitly choose **Run with PowerShell**.

As mentioned previously, the full details of the shell APIs are out of scope for this book; as you might expect, the full picture is not quite as simple as I've shown here.

System Processes

Throughout this and the preceding chapter, I've alluded to various processes that run with higher privileges than a normal user. This is because, even when no user is logged in to the operating system, the system still needs to perform tasks like waiting for authentication, managing hardware, and communicating over the network.

The kernel could perform some of these tasks. However, writing kernel code is more difficult than user-mode code, for a number of reasons: the kernel doesn't have as wide a range of APIs available; it's resource constrained, especially in terms of memory; and any coding mistake could result in the system crashing or being exposed to a security vulnerability.

To avoid these challenges, Windows runs a variety of processes outside of kernel mode, with a high privilege level, to provide important facilities. We'll go through some of these special processes in this section.

The Session Manager

The *Session Manager Subsystem (SMSS)* is the first user-mode process started by the kernel after boot. It's responsible for setting up the working environment for subsequent processes. Some of its responsibilities include:

- Loading known DLLs and creating the *Section* objects
- Starting subsystem processes such as CSRSS
- Initializing base DOS devices such as serial ports
- Running automatic disk integrity checks

The Windows Logon Process

The *Windows logon* process is responsible for setting up a new console session, as well as displaying the logon user interface (primarily through the LogonUI application). It's also responsible for starting the *user-mode font driver (UMFD)* process, which renders fonts to the screen, and the *desktop window manager (DWM)* process, which performs desktop compositing operations to allow for fancy, transparent windows and modern GUI touches.

The Local Security Authority Subsystem

I've already mentioned LSASS in the context of the SRM. However, it's worth stressing its important role in authentication. Without LSASS, a user would not be able to log on to the system. We'll cover LSASS's roles and responsibilities in much more detail in [Chapter 10](#).

The Service Control Manager

The *service control manager (SCM)* is responsible for starting most privileged system processes on Windows. It manages these processes, referred to as *services*, and can start and stop them as needed. For example, the SCM could start a service based on certain conditions, such as a network becoming available.

Each service is a securable resource with fine-grained controls determining which users can manipulate its state. By default, only an administrator can manipulate a service. The following are some of the most important services running on any Windows system:

Remote Procedure Call Subsystem (RPCSS) The RPCSS service manages the registration of remote procedure call endpoints, exposing the registration to local clients as well as over the network. This service is essential to a running system; in fact, if this process crashes, it will force Windows to reboot.

DCOM Server Process Launcher The DCOM Server Process Launcher is a counterpart to RPCSS (and used to be part of the same service). It's used to start Component Object Model (COM) server processes on behalf of local or remote clients.

Task Scheduler Being able to schedule an action to run at a specific time and date is a useful feature of an operating system. For example, perhaps you want to ensure that you delete unused files on a specific schedule. You could set up an action with the Task Scheduler service to run a cleanup tool on that schedule.

Windows Installer This service can be used to install new programs and features. By running as a privileged service, it permits installation and modification in normally protected locations on the filesystem.

Windows Update Having a fully up-to-date operating system is crucial to the security of your Windows system. When Microsoft releases new security fixes, they should be installed as soon as possible. To avoid requiring the user to check for updates, this service runs in the background, waking up periodically to check the internet for new patches.

Application Information This service provides a mechanism for switching between an administrator and non-administrator user on the same desktop. This feature is usually referred to as *User Account Control* (UAC). You can start an administrator process by using the `runas` verb with the shell APIs. We'll cover how UAC works under the hood in the next chapter.

We can query the status of all services controlled by the SCM using various tools. PowerShell has the built-in `Get-Service` command; however, the PowerShell module used in this book provides a more comprehensive command, `Get-Win32Service`, that can inspect the configured security of a service as well as additional properties not exposed using the default command. [Listing 3-24](#) shows how to query for all current services.

```
PS> Get-Win32Service
Name           Status    ProcessId
----           -
AarSvc         Stopped   0
AESMSvc       Running  7440
AJRouter       Stopped   0
ALG            Stopped   0
AppIDSvc       Stopped   0
Appinfo        Running  8460
--snip--
```

Listing 3-24: Displaying all services using `Get-Win32Service`

The output shows the name of the service, its status (either `Stopped` or `Running`), and, if it's running, the process ID of the service process. If you list the service's properties using `Format-List`, you'll also be able to see additional information, such as a full description of the service.

Worked Examples

Let's walk through some worked examples to practice using the various commands covered in this chapter for security research or systems analysis.

Finding Executables That Import Specific APIs

At the beginning of this chapter, you saw how to use the `Get-Win32ModuleImport` command to extract an executable file's imported APIs. One use for this command that I find especially helpful when I'm trying to track down security issues is identifying all the executables that use a particular API, such as `CreateProcess`, and then using this list to reduce the files I need to reverse engineer. You can perform such a search with the basic PowerShell script shown in [Listing 3-25](#).

```
PS> $imps = ls "$env:WinDir\*.exe" | ForEach-Object {  
    Get-Win32ModuleImport -Path $_.FullName  
}  
PS> $imps | Where-Object Names -Contains "CreateProcessW" |  
Select-Object ModulePath  
ModulePath  
-----  
C:\WINDOWS\explorer.exe  
C:\WINDOWS\unins000.exe
```

Listing 3-25: Finding executables that import CreateProcess

Here, we start by enumerating all the `.exe` files in the `Windows` directory. For every executable file, we call the `Get-Win32ModuleImport` command. This will load the module and parse its imports. This can be a time-consuming process, so it's best to capture the results into a variable, as we do here.

Next, we select only the imports that contain the `CreateProcessW` API. The `Names` property is a list containing the imported names for a single DLL. To get the resulting list of executable files that import a specific API, we can select the `ModulePath` property, which contains the original loaded pathname.

You can use the same technique to enumerate DLL files or drivers and quickly discover targets for reverse engineering.

Finding Hidden Registry Keys or Values

In "Listing the Registry's Contents" on page 81, I mentioned that one of the big advantages of using the native system calls over the Win32 APIs to interact with the registry is that they allow you to access keys and values with NUL characters in their names. It would be useful to be able to find these keys and values so you can try to detect software on your system that is actively trying to hide registry keys or values from the user (some malware families, such as Kovter and Poweliks, are known to use this technique). Let's start by finding keys with NUL characters in the name ([Listing 3-26](#)).


```

PS> $key = New-NtKey -Win32Path "HKCU\SOFTWARE\`0HIDDENKEY"
PS> ls NtKeyUser:\SOFTWARE -Recurse | Where-Object Name -Match "`0"
Name                                     TypeName
----                                     -
SOFTWARE\ HIDDENKEY Key

PS> Remove-NtKey $key
PS> $key.Close()

```

Listing 3-26: Finding hidden registry keys

We first create a key in the current user's hive with a NUL character in it. If you try to find this key using the built-in registry provider, it will fail. Instead, we do a recursive listing of the current user's hive and select any keys that have a NUL character in the name. In the output, you can see that the hidden key was discovered.

To find hidden values, we can query the list of values of a key by enumerating its `Values` property. Each value contains the name of the key and the data value ([Listing 3-27](#)).

```

❶ PS> $key = New-NtKey -Win32Path "HKCU\SOFTWARE\ABC"
PS> Set-NtKeyValue -Key $key -Name "`0HIDDEN" -String "HELLO"
❷ PS> function Select-HiddenValue {
    [CmdletBinding()]
    param(
        [parameter(ValueFromPipeline)]
        $Key
    )

    Process {
        ❸ foreach($val in $Key.Values) {
            if ($val.Name -match "`0") {
                [PSCustomObject]@{
                    RelativePath = $Key.RelativePath
                    Name = $val.Name
                    Value = $val.DataObject
                }
            }
        }
    }
}
❹ PS> ls -Recurse NtKeyUser:\SOFTWARE | Select-HiddenValue | Format-Table
RelativePath Name      Value
-----
SOFTWARE\ABC HIDDEN HELLO

PS> Remove-NtKey $key
PS> $key.Close()

```

Listing 3-27: Finding hidden registry values

We start by creating a normal key, then adding a value with a NUL character in the name ❶. We then define a function, `Select-HiddenValue` ❷, that will check keys in the pipeline and select any value with a NUL character in the name, returning a custom object to the pipeline ❸.

Next, we recursively enumerate the current user's hive and filter the keys through the `Select-HiddenValue` function ❹. You can see in the output that we discovered the hidden value.

Wrapping Up

This chapter provided a quick tour through the Windows user-mode components. We started with a dive into Win32 APIs and the loading of DLLs. Understanding this topic is important, as it reveals how user-mode applications communicate with the kernel and implement common features.

Next, I provided an overview of the Win32 GUI, including a description of the separate system call table used for *WIN32K*, which is the kernel-mode component of the Win32 subsystem. I introduced the window station and desktop object types and outlined the purpose of the console session, as well as how it corresponds to the desktop you see as a user.

I then returned to the topic of Win32 APIs by detailing the differences and similarities between a Win32 API (in this case, `CreateMutexEx`) and the underlying system call (`NtCreateMutant`). This discussion should have given you a better understanding of how the Win32 APIs interact with the rest of the operating system. I also introduced the differences between DOS device paths and native paths as understood by a system call, a topic that is important for understanding how user-mode applications interact with the filesystem.

I concluded with a discussion of several topics related to Win32 processes and threads, covering the APIs used to create processes directly or through the shell and providing an overview of well-known system processes. In later chapters, we'll revisit many of these topics in more depth. In the next three chapters, we'll focus on how Windows implements security through the SRM.